



Material didáctico — Proyecto Intermodular

Sergi Garcia Barea

CC BY-SA 4.0

Material didáctico — Proyecto Intermodular

Guia Didactica Proyecto 1 Index

 *Lo primero: aprender a planificar antes de programar*

Proyecto 1 no es solo escribir código. Es aprender a **pensar** antes de construir: identificar un problema, definir requisitos, elegir tecnologías, comunicar ideas y documentar el proceso. Todo lo que aprendas aquí te servirá en Proyecto 2 y en tu vida profesional.

¿Qué es Proyecto 1?

Proyecto 1 es la primera fase del módulo de Proyecto. Se desarrolla durante el primer curso y consta de **7 unidades didácticas** que te guían desde la idea inicial hasta la preparación de una propuesta de aplicación web.

Las 7 unidades

UD	Qué harás	Enfoque
 UD 1.1: MVP	Definir qué es un producto mínimo viable	Conceptos clave
 UD 1.2: SCRUM Lite	Metodología ágil adaptada a estudiantes	Organización
 UD 1.3: Requisitos	Capturar y documentar qué necesita el sistema	Análisis
 UD 1.4: Tecnologías	Elegir stack tecnológico con criterio	Decisiones
 UD 1.5: Comunicación	Presentar y defender tu proyecto	Soft skills
 UD 1.6: Documentación	Crear documentación técnica moderna	Docs as Code
 UD 1.7: Propuesta	Ejemplos de proyectos y cómo empezar	Práctica

Evaluación

Cada UD se evalúa de forma independiente. Consulta la guía de evaluación para conocer los criterios y pesos de cada unidad.

Competencias Proyecto 1

Competencias generales

- **CG1:** Capacidad de análisis y síntesis para identificar problemas y proponer soluciones
- **CG2:** Capacidad de organización y planificación del trabajo
- **CG3:** Comunicación oral y escrita en la presentación de resultados
- **CG4:** Trabajo en equipo y colaboración con otros compañeros
- **CG5:** Aprendizaje autónomo y búsqueda de información técnica

- **CE1:** Definir un Producto Mínimo Viable (MVP) con alcance claro
- **CE2:** Aplicar metodología ágil (SCRUM Lite) en un proyecto individual
- **CE3:** Capturar y documentar requisitos e historias de usuario
- **CE4:** Elegir tecnologías con criterio arquitectónico
- **CE5:** Comunicar y defender un proyecto ante un tribunal
- **CE6:** Crear documentación técnica con enfoque Docs as Code
- **CE7:** Diseñar y proponer una aplicación web con alcance claro y persistencia de datos

Relación con las asignaturas

Asignatura	Competencias que aporta
Programación	Lógica, estructuras de datos, POO
Bases de Datos	Diseño y consulta de bases de datos
Lenguajes de Marcas	HTML, XML, documentación
Entornos de Desarrollo	Git, testing, depuración
Sistemas Informáticos	Instalación y configuración

Evaluación Proyecto 1

Criterios de evaluación

Criterio	Peso orientativo
Cumplimiento de objetivos	30%
Calidad técnica del trabajo	25%
Documentación y memoria	20%
Presentación y defensa	15%
Trabajo en equipo (si aplica)	10%

Sistema de calificación

Cada UD se califica de 0 a 10. La nota final es la media ponderada:

UD	Peso orientativo
UD 1.1: MVP	15%
UD 1.2: SCRUM Lite	10%
UD 1.3: Requisitos	15%
UD 1.4: Tecnologías	15%
UD 1.5: Comunicación	15%
UD 1.6: Documentación	10%
UD 1.7: Propuesta	20%

Nota: los pesos son orientativos; el profesor concretará los porcentajes al inicio del curso.

Criterios de calidad

Para obtener más de un 5 en cada UD:

- **UD 1.1:** MVP viable, alcance definido, justificación clara
- **UD 1.2:** Tablero funcional, sprints planificados, retrospectiva
- **UD 1.3:** Requisitos completos, historias con criterios de aceptación
- **UD 1.4:** Stack justificado, arquitectura por capas, decisiones documentadas
- **UD 1.5:** Presentación clara, demo funcional, defensa solvente

- **UD 1.6:** README completo, documentación técnica, código comentado
- **UD 1.7:** Proyecto funcional, CRUD completo, persistencia en BD

Observaciones importantes

- Las UD's se entregan en las fechas indicadas
- La **falta de entrega** de una UD supone no superar la evaluación continua
- Las faltas de ortografía en la documentación penalizan
- El código debe ser original (no copiado de internet sin atribución)

Guia Didactica Proyecto 1 Uds U1 1 Mvp

 *MVP: la versión más pequeña que aún tiene sentido*

Un MVP no es un proyecto a medias. Es la versión más pequeña de tu idea que **resuelve un problema real**. Si no puedes explicar tu MVP en una frase, es demasiado grande. Esta unidad te enseña a encontrar esa versión mínima que tiene sentido.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Definir qué es un MVP y por qué importa
-  Identificar un problema real que merezca la pena resolver
-  Listar y priorizar funcionalidades con criterio
-  Cortar el alcance sin miedo para tener algo terminado

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué es un MVP?	Definición, origen y mitos
02	Identificar el problema	Cómo encontrar problemas reales
03	Definir features	Listar, clasificar y priorizar
04	Cortar sin piedad	Reglas de alcance y cuándo parar
05	Ejemplos reales	Ideas que funcionaron y por qué
06	Errores comunes	Los 7 pecados del MVP
07	Herramientas	Qué usar para organizar tu MVP
08	Template	Plantilla rellenable para tu proyecto
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y empieza a rellenar
- **Si no tienes idea:** empieza por el [punto 02 \(identificar problema\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

Guia Didactica Proyecto 1 Uds U1 2 Scrum Lite

 **SCRUM:** trabajar en equipo sin morir en el intento

Tener una buena idea (UD 1.1) no sirve de nada si no sabes **cómo organizar el trabajo**. SCRUM es un método que obliga a trabajar en ciclos cortos, terminar cosas de verdad y hablar entre el equipo. Esta unidad es “SCRUM Lite”: lo esencial sin la burocracia de las empresas grandes.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

- 🏃 Entender qué es SCRUM y por qué funciona para proyectos reales
- 📅 Organizar tu trabajo en sprints con objetivos claros
- 📊 Usar un tablero Kanban para ver qué está hecho y qué falta
- 📝 Escribir user stories que tengan sentido para el usuario
- 🎯 Estimar esfuerzo sin experiencia previa
- 🚫 Evitar los errores más comunes de SCRUM en el aula

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué es ágil?	Waterfall vs Agile, el Manifiesto
02	¿Qué es SCRUM?	Roles, ceremonias, artefactos
03	El Sprint	Ciclos de 1-2 semanas con objetivo
04	Tablero Kanban	To Do → In Progress → Done
05	User Stories	“Como... quiero... para que...”
06	Estimación	T-shirt sizes, Planning Poker
07	Errores comunes	Los 6 pecados de SCRUM en el aula
08	Template	Planning, checklist y retrospectiva
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y empieza a rellenar
- **Si no tienes idea:** empieza por el [punto 02 \(¿Qué es SCRUM?\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

Guia Didactica Proyecto 1 Uds U1 3 Requisitos

 *Los requisitos: saber qué construir antes de construirlo*

Programar sin requisitos es como cocinar sin receta: quizá salga algo comestible, pero seguramente no es lo que querías. Esta unidad te enseña a **definir qué debe hacer tu app**, para quién y con qué restricciones, antes de escribir una sola línea de código.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Diferenciar requisitos funcionales y no funcionales
-  Escribir historias de usuario claras y testables
-  Definir criterios de aceptación en formato Dado/Cuando/Entonces
-  Priorizar requisitos con MoSCoW, Kano y valor vs esfuerzo
-  Evitar los errores más comunes en definición de requisitos
-  Generar un documento de requisitos completo para tu proyecto

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué son los requisitos?	Funcionales vs no funcionales
02	Requisitos funcionales	Qué debe hacer la app
03	Requisitos no funcionales	Rendimiento, seguridad, usabilidad
04	Historias de usuario	“Como... quiero... para que...”
05	Criterios de aceptación	Dado/Cuando/Entonces
06	Priorización	MoSCoW, Kano, valor vs esfuerzo
07	Errores comunes	“El usuario puede hacer todo” y más
08	Template requisitos	Documento listo para usar
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y rellena el documento
- **Si no tienes idea:** empieza por el [punto 01 \(¿Qué son los requisitos?\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 1.4: Elección de Tecnologías

 *Elegir bien ahorra semanas de sufrimiento*

Cada proyecto necesita tecnologías: un lenguaje, una base de datos, un framework. Pero elegir mal puede costarte semanas de refactorización o un producto que no funciona. Esta unidad te enseña a **evaluar opciones con criterio**, no por moda ni por lo que usa tu amigo.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

- 🗨 Entender por qué la elección tecnológica importa y cuál es el coste de equivocarse
- 📊 Evaluar tecnologías con criterios objetivos (rendimiento, curva de aprendizaje, ecosistema, comunidad)
- 🔍 Comparar lenguajes y frameworks: Python, Java, JavaScript, PHP
- 💾 Elegir la base de datos correcta: SQL vs NoSQL, SQLite, PostgreSQL, MongoDB
- ⚖ Decidir entre frontend y backend, MVC, monolito vs microservicios
- 📋 Comparar stacks completos: LAMP, MERN, Python+SQLite, Java+MySQL
- ✍ Redactar la justificación técnica de tu proyecto
- 📄 Documentar tu stack con el template oficial

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Por qué elegir?	El coste de elegir mal
02	Criterios de elección	Qué valorar antes de decidir
03	Lenguajes y frameworks	Python, Java, JavaScript, PHP
04	Bases de datos	SQL vs NoSQL, cuándo usar cada una
05	Frontend vs Backend	MVC, monolito, arquitectura
06	Comparativa stack	LAMP, MERN, Python+SQLite, Java+MySQL
07	Justificación	Cómo argumentar tus decisiones
08	Template stack	Documento listo para usar
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y documenta tu stack
- **Si no sabes qué elegir:** empieza por el [punto 02 \(criterios\)](#) para aprender a evaluar

- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 1.5: Comunicación

 *Comunicar es tan importante como programar*

Puedes tener el mejor proyecto del mundo, pero si no sabes explicarlo, el tribunal no lo valorará. Esta unidad te enseña a **presentar tu trabajo** de forma clara, profesional y convincente: pitch de 5 minutos, diapositivas efectivas, demo en vivo y defensa ante preguntas del tribunal.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Preparar un pitch de 5 minutos con estructura clara
-  Modular tu lenguaje verbal: tono, ritmo y confianza
-  Gestionar tu lenguaje no verbal: postura, gestos, contacto visual
-  Escribir un memoria/proyecto clara y bien estructurada
-  Crear diapositivas efectivas (menos texto, más impacto)
-  Hacer una demo en vivo sin problemas
-  Responder preguntas del tribunal con seguridad

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Por qué comunicar?	La presentación es tu primera impresión
02	Pitch 5 minutos	Estructura: problema, solución, demo, tech, cierre
03	Lenguaje verbal	Tono, ritmo, muletillas y confianza
04	Lenguaje no verbal	Postura, contacto visual, gestos
05	Memoria del proyecto	Estructura y estilo del documento
06	Diapositivas	Menos texto, más visuales, herramientas
07	Demo práctica	Preparar, probar y plan B
08	Preguntas del tribunal	Preguntas frecuentes y cómo responder
09	Cierre	Mini-chequeo, vocabulario y resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 02 \(pitch\)](#) y ensaya
- **Si no tienes idea:** empieza por el [punto 01 \(¿Por qué comunicar?\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09



UD 1.6: Documentación

 *Documentar no es opcional, es parte del proyecto*

Un proyecto sin documentación es como un libro sin índice: puede ser genial, pero nadie sabe cómo usarlo. Esta unidad te enseña a **escribir documentación clara, útil y profesional** que haga que tu proyecto sea entendible por otros — y por ti mismo dentro de 6 meses.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender la filosofía Docs as Code y por qué importa
-  Escribir un README claro y completo
-  Crear documentación técnica: arquitectura, setup, API
-  Redactar un manual de usuario para no técnicos
-  Usar Markdown con soltura
-  Elegir las herramientas de documentación adecuadas
-  Evitar los errores más comunes al documentar
-  Usar una plantilla para documentar tu proyecto

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Por qué documentar?	La documentación como parte del desarrollo
02	README	La puerta de entrada a tu proyecto
03	Doc técnica	Arquitectura, setup y API
04	Manual de usuario	Escribir para no técnicos
05	Markdown	Sintaxis, formato y consejos
06	Herramientas doc	GitBook, Notion, draw.io y más
07	Errores de documentación	“Lo documento luego”, demasiado técnico, demasiado vago
08	Plantilla de doc	Template listo para usar
09	Cierre	Mini-chequeo, vocabulario y resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(plantilla\)](#) y adapta
- **Si no tienes idea:** empieza por el [punto 01 \(¿Por qué documentar?\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 1.7: Propuesta de Proyecto

 *Antes de programar, hay que decidir qué hacer*

Tienes las habilidades de comunicación y documentación. Ahora toca **definir qué vas a construir**. La propuesta de proyecto es el documento que recoge tu idea, la justifica, la define y la plantea como un plan de acción realista. Es el contrato entre tu equipo y el tribunal.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué es una propuesta de proyecto y para qué sirve
-  Leer y analizar un briefing de proyecto
-  Generar ideas originales mediante técnicas de brainstorming
-  Seleccionar la mejor idea según criterios claros
-  Definir el alcance de tu proyecto (qué entra y qué no)
-  Crear un cronograma realista con hitos
-  Redactar una propuesta completa y profesional
-  Evitar los errores más comunes en propuestas

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué es una propuesta?	Estructura, purpose y formato
02	Briefing	Cómo leer y entender el encargo
03	Ideas de proyecto	Técnicas de brainstorming
04	Selección de idea	Criterios y viabilidad
05	Definición de alcance	Qué entra y qué no
06	Cronograma	Hitos, deadlines y planificación
07	Ejemplo de propuesta	Propuesta completa de ejemplo
08	Errores de propuesta	Demasiado ambicioso, demasiado vago
09	Cierre	Mini-chequeo, vocabulario y resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 07 \(ejemplo\)](#) y úsalo como referencia
- **Si no tienes idea:** empieza por el [punto 02 \(briefing\)](#) para entender qué te piden
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

Guia Didactica Proyecto 2 Index

La segunda oportunidad de hacerlo mejor

Ya tienes experiencia de Proyecto 1: sabes lo que es planificar mal, lo que es no llegar a tiempo, lo que es tener un código que da vergüenza enseñar. Ahora sabes mejor lo que NO hacer. Y eso te da ventaja.

Proyecto 2 es la continuación del módulo. Partes de lo aprendido en Proyecto 1 y lo conviertes en un **producto completo y profesional**: aplicación web Full-Stack, desplegada, monitorizada y con procesos de calidad.

Las 10 unidades

UD	Qué harás	Enfoque
 UD 2.1: Requisitos Avanzado	Refinamiento y criterios de aceptación	Análisis
 UD 2.2: SCRUM Avanzado	Métricas, burndown, retrospectivas	Metodología
 UD 2.3: Git Avanzado	GitFlow, hooks, CI/CD básico	Control versiones
 UD 2.4: Testing	Tipos de test, herramientas, TDD	Calidad
 UD 2.5: Patrones de Diseño	Patrones creacionales, estructurales, comportamiento	Arquitectura
 UD 2.6: Diagramas	C4, UML esencial, secuencia, infraestructura	Modelado
 UD 2.7: IA como Copiloto	LLMs, prompt engineering, testing con IA	Herramientas
 UD 2.8: Despliegue	Docker, CI/CD, zero-downtime	DevOps
 UD 2.9: Monitorización	Prometheus, Grafana, logs estructurados	Operaciones
 UD 2.10: Propuesta Final	Ejemplos de proyectos Full-Stack	Práctica

Evaluación

Cada UD se evalúa de forma independiente. Consulta la guía de evaluación para conocer los criterios y pesos de cada unidad.

Competencias generales

- **CG1:** Capacidad para evolucionar y mantener software existente
- **CG2:** Integración de tecnologías y servicios externos
- **CG3:** Automatización de procesos (testing, despliegue)
- **CG4:** Trabajo en equipo avanzado (roles, metodologías ágiles)
- **CG5:** Compromiso con la calidad y la seguridad

Competencias específicas

- **CE1:** Refinar requisitos con criterios de aceptación e INVEST
- **CE2:** Aplicar SCRUM avanzado con métricas y retrospectivas
- **CE3:** Usar Git con estrategias de branching (GitFlow, trunk-based)
- **CE4:** Asegurar la calidad con tests unitarios, de integración y manuales
- **CE5:** Aplicar patrones de diseño en arquitectura web
- **CE6:** Modelar sistemas con diagramas C4 y UML
- **CE7:** Integrar IA como herramienta de desarrollo
- **CE8:** Desplegar aplicaciones con Docker y CI/CD
- **CE9:** Monitorizar aplicaciones con Prometheus y Grafana
- **CE10:** Desarrollar un proyecto Full-Stack completo

Relación con las asignaturas de 2º

Asignatura	Competencias que aporta
Desarrollo Web en Entorno Cliente	Frontend avanzado, frameworks
Desarrollo Web en Entorno Servidor	APIs, seguridad, despliegue
Despliegue de Aplicaciones Web	CI/CD, cloud, contenedores
Diseño de Interfaces	UX, usabilidad, prototipado
Empresa	Modelos de negocio, emprendimiento

Entregables por UD

UD 2.1: Requisitos Avanzado

- Backlog priorizado con historias INVEST
- Criterios de aceptación para cada historia
- Estimación en puntos de historia

UD 2.2: SCRUM Avanzado

- Velocity del equipo (o proyección)
- Burndown chart del sprint
- Retrospectiva documentada

UD 2.3: Git Avanzado

- Estrategia de branching definida
- Hooks de Git configurados
- Pipeline CI básico funcionando

UD 2.4: Testing

- Tests unitarios de las funciones core
- Un test de integración del flujo principal
- Protocolo de pruebas manuales documentado
- Checklist de calidad completado antes de cada entrega

UD 2.5: Patrones de Diseño

- Documento de arquitectura con patrones aplicados
- Justificación de cada patrón elegido
- Código refactorizado según patrones

UD 2.6: Diagramas

- Diagrama de contexto (C4 nivel 1)
- Diagrama de componentes o contenedores

- Diagrama ER de la base de datos
- Al menos 1 diagrama de secuencia

UD 2.7: IA como Copiloto

- Documento con prompts utilizados
- Ejemplos de código generado y adaptado
- Reflexión sobre uso de IA

UD 2.8: Despliegue

- Dockerfile funcional
- docker-compose.yml (si aplica)
- CI/CD configurado (GitHub Actions)
- App desplegada con HTTPS

UD 2.9: Monitorización

- Logs estructurados en la aplicación
- Endpoint /metrics funcionando
- Dashboard básico en Grafana

UD 2.10: Propuesta Final

- Código fuente completo y organizado
- Aplicación desplegada y accesible
- Documentación técnica completa
- Memoria del proyecto
- Presentación y demo

Formato de entrega

Todos los entregables se suben al repositorio de GitHub. La memoria y documentación pueden estar en Markdown dentro del repositorio o en PDF.



Criterios de evaluación

Criterio	Peso orientativo
Funcionalidad y requisitos	25%
Calidad técnica y arquitectura	20%
Testing y calidad	15%
Despliegue y DevOps	15%
Documentación y memoria	10%
Monitorización	5%
Presentación y defensa	10%

Sistema de calificación

Cada UD se califica de 0 a 10. La nota final es la media ponderada:

UD	Peso orientativo
UD 2.1: Requisitos Avanzado	10%
UD 2.2: SCRUM Avanzado	10%
UD 2.3: Git Avanzado	10%
UD 2.4: Testing	10%
UD 2.5: Patrones de Diseño	10%
UD 2.6: Diagramas	10%
UD 2.7: IA como Copiloto	5%
UD 2.8: Despliegue	15%
UD 2.9: Monitorización	10%
UD 2.10: Propuesta Final	20%

Nota: los pesos son orientativos; el profesor concretará los porcentajes al inicio del curso.

Criterios de calidad

Para obtener más de un 5 en cada UD:

- **UD 2.1:** Historias con INVEST, criterios de aceptación, backlog priorizado
- **UD 2.2:** Velocity calculada, burndown chart, retrospectiva efectiva
- **UD 2.3:** Estrategia de branching clara, hooks configurados, CI funcionando
- **UD 2.4:** Tests unitarios del core, integración del flujo principal, pruebas manuales antes de entregar
- **UD 2.5:** Patrones aplicados correctamente, justificación de decisiones
- **UD 2.6:** Diagrama C4 mínimo, ER completo, al menos 1 secuencia
- **UD 2.7:** Uso responsable de IA, prompts efectivos, código entendido
- **UD 2.8:** Docker funcional, CI/CD configurado, app desplegada con HTTPS
- **UD 2.9:** Logs estructurados, métricas básicas, dashboard funcional
- **UD 2.10:** Proyecto Full-Stack completo, documentado, desplegado

Observaciones

- La **evolución** respecto a Proyecto 1 se valora positivamente
- El despliegue en producción (aunque sea gratuito) suma puntos
- Los tests automatizados son obligatorios en Proyecto 2
- La memoria debe incluir la comparativa con Proyecto 1

UD 2.1: Requisitos Avanzado

 *Requisitos al nivel siguiente: de vagos a verificables*

En Proyecto 1 aprendiste a escribir historias de usuario y criterios de aceptación. Ahora toca **eleva el nivel**: estimar con story points, refinar el backlog como un equipo profesional y establecer trazabilidad entre requisitos y código.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Evaluar historias de usuario con los criterios **INVEST**

- 📊 Priorizar con **MoSCoW avanzado** y el modelo **Kano**
- 🎲 Estimar con **story points** y **Planning Poker**
- 🔍 Realizar **sesiones de refinamiento** de backlog
- ✍️ Escribir criterios de aceptación con **Given/When/Then** y casos extremos
- 📐 Construir una **matriz de trazabilidad** de requisitos
- 🚫 Identificar y evitar **errores avanzados** de requisitos

Mapa de la unidad

Punto	Tema	Idea clave
01	Criterios INVEST	Las 6 cualidades de una buena historia
02	MoSCoW avanzado y Kano	Priorización con criterio, no por capricho
03	Story points y Planning Poker	Estimar esfuerzo relativo sin morir en el intento
04	Refinamiento de backlog	La sesión que salva sprints
05	Criterios de aceptación avanzado	Given/When/Then y casos extremos
06	Trazabilidad	De requisito a código y viceversa
07	Errores avanzados	Los 7 pecados capitales de los requisitos
08	Template avanzado	Documento listo para Proyecto 2
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y adapta el documento a tu proyecto
- **Si no tienes idea:** empieza por el [punto 01 \(INVEST\)](#) para entender qué hace buena a una historia
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

 *SCRUM no es solo tablero Kanban. Es un sistema de mejora continua*

En Proyecto 1 aprendiste lo básico de SCRUM: sprints, Kanban, user stories. Ahora toca **eleva el nivel**: medir la velocidad real del equipo, detectar bloqueos antes de que te destruyan, hacer retrospectivas que sirvan de verdad y entender el papel del SCRUM Master como facilitador, no como jefe.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Usar **Planning Poker** como técnica formal de estimación
-  Medir y usar la **velocidad del equipo** para planificar
-  Interpretar gráficos de **burndown y burnup**
-  Detectar y resolver **bloqueos y dependencias** antes de que te destruyan
-  Liderar **retrospectivas avanzadas** que generen mejoras reales
-  Definir **métricas y KPIs** para medir la salud del proyecto
-  Ejercer el rol de **SCRUM Master** como estudiante

Mapa de la unidad

Punto	Tema	Idea clave
01	Planning Poker	Estimación colectiva con cartas
02	Velocidad del equipo	Points por sprint para predecir
03	Burndown y burnup	Gráficos que muestran el progreso real
04	Bloqueos y dependencias	Qué hacer cuando algo se atasca
05	Retrospectivas avanzadas	Más allá de “¿qué salió bien?”
06	Métricas y KPIs	Qué medir y cómo usar los datos
07	SCRUM Master estudiante	Tu rol como facilitador del equipo
08	Template SCRUM avanzado	Planning, retrospectiva y métricas
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y adapta los formatos a tu proyecto
- **Si no tienes idea:** empieza por el [punto 01 \(Planning Poker\)](#) para entender la estimación formal
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.3: Git Avanzado

 *Git no es solo hacer commit. Es una estrategia de trabajo*

En Proyecto 1 aprendiste lo básico: commit, push, pull, branch. Ahora toca **dominar Git**: estrategias de branching como GitFlow y Trunk-Based, hooks que

automatizan calidad, CI/CD con GitHub Actions y convenciones de commit que hacen tu historial legible.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

- 🌱 Aplicar la estrategia **GitFlow** para gestionar features y releases
- 🚀 Entender **Trunk-Based Development** y cuándo usarlo
- 🔗 Resolver **conflictos de merge** sin morir en el intento
- 🔒 Configurar **Git hooks** (pre-commit, commit-msg) con código
- 🤖 Configurar **CI/CD** con GitHub Actions con código real
- 📄 Usar **Conventional Commits** para un historial limpio
- 👥 Hacer **code review** efectiva

Mapa de la unidad

Punto	Tema	Idea clave
01	GitFlow	Branches para features, releases y hotfixes
02	Trunk-Based	Desarrollo directo en main con branches cortos
03	Conflictos	Cómo resolverlos sin perder trabajo
04	Git Hooks	Automatizar calidad antes del commit
05	CI/CD	GitHub Actions para build, test y deploy
06	Convenciones de commit	Conventional Commits para historial limpio
07	Code Review	Cómo revisar código sin destruir relaciones
08	Template Git	Workflow listo para tu proyecto
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y adapta el workflow a tu proyecto
 - **Si no tienes idea:** empieza por el [punto 01 \(GitFlow\)](#) para entender la estrategia de branches
 - **Si quieres entender bien:** ve los puntos en orden del 01 al 09
-

UD 2.4: Testing

 *El seguro de tu proyecto: que no falle cuando más importa*

En Proyecto 1 tu app funcionaba “en tu ordenador”. En Proyecto 2 toca demostrar que funciona **en cualquier ordenador, cualquier día, delante del tribunal**. El testing es la herramienta que te da esa certeza: automatizada, verificable y documentada.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Explicar **por qué los tests** son esenciales para tu proyecto
-  Distinguir **unitario, integración, E2E y manual** y cuándo usar cada uno
-  **Priorizar** qué probar con las 3 reglas (uso, fragilidad, cambio)
-  Integrar tests en tu **flujo de trabajo** (TDD, pre-commit, CI)
-  Aplicar un **protocolo de pruebas manuales** antes de cada entrega
-  Elegir las **herramientas** adecuadas a tu stack (Vitest, JUnit, Playwright)
-  Compartir la **responsabilidad del testing** con tu equipo
-  Usar un **checklist de calidad** completo antes del tribunal

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Por qué hacer tests?	Los tests son un seguro, no un trámite
02	Tipos de test	Unitario, integración, E2E y manual
03	Qué probar en tu proyecto	Prioriza: lo que más usa, rompe y cambia
04	Tests en tu flujo	TDD, pre-commit y CI sin morir en el intento
05	Pruebas manuales	La red de seguridad antes de cada entrega
06	Herramientas de testing	Vitest, JUnit, Playwright y Testing Library
07	Testing en equipo	Code review, acuerdos y pair testing
08	Checklist de calidad	Todo lo que verificar antes de entregar
09	Cierre	Retos, laboratorio y preguntas para pensar

Competencias que desarrollarás

Competencia	Cómo se trabaja
Pensamiento crítico	Decidir qué probar y qué no, priorizar esfuerzo
Resolución de problemas	Detectar y diagnosticar fallos con tests
Trabajo en equipo	Compartir la responsabilidad de la calidad
Comunicación técnica	Documentar comportamiento con tests legibles
Autonomía	Saber que tu código funciona antes de entregarlo

► 🕒 ¿Cuánto tiempo dedicar al testing en el proyecto?

¿Por dónde empiezo?

- **Si tienes prisa:** ve al [punto 08 \(checklist\)](#) y aplica lo mínimo antes de la próxima entrega

- **Si no has escrito un test nunca:** empieza por el [punto 01](#) y el [punto 02](#)
- **Si quieres entender el testing de verdad:** sigue el orden del 01 al 09

UD 2.5: Patrones de Diseño

 *No reinventes la rueda: usa patrones que ya funcionan*

Los **patrones de diseño** son soluciones probadas a problemas que aparecen una y otra vez en el desarrollo de software. No son código copiado: son **plantillas de diseño** que te ayudan a crear código más limpio, flexible y mantenible. Conocerlos te hace programador mejor.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué son los patrones de diseño y por qué existen
-  Usar patrones **creacionales**: Singleton, Factory, Builder
-  Usar patrones **estructurales**: Adapter, Facade, Composite
-  Usar patrones **comportamiento**: Observer, Strategy, Command
-  Entender **MVC** y **Repository** como arquitecturas completas
-  Saber **cuándo usar** cada patrón (y cuándo no)

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué son los patrones?	Soluciones reutilizables a problemas comunes
02	Patrones creacionales	Singleton, Factory, Builder
03	Patrones estructurales	Adapter, Facade, Composite
04	Patrones de comportamiento	Observer, Strategy, Command
05	MVC	Model-View-Controller explicado
06	Repository	Separar acceso a datos de la lógica
07	Cuándo usar cada patrón	Guía práctica de selección
08	Template patrones	Selección de patrón para tu proyecto
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 07 \(cuándo usar\)](#) para ver qué patrón necesitas
- **Si no tienes idea:** empieza por el [punto 01 \(¿Qué son?\)](#) para entender el concepto
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.6: Diagramas

 *Un diagrama dice más que mil líneas de código*

Los diagramas no son burocracia: son la **forma más rápida de comunicar** cómo funciona tu sistema. Un diagrama de secuencia explica un flujo en 30 segundos. Un diagrama de componentes muestra la arquitectura en un vistazo. Son la documentación visual que tu proyecto necesita.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

- 📖 Entender qué son los diagramas y por qué son importantes
- 🏗️ Usar el **modelo C4** para documentar arquitectura en 4 niveles
- 🗃️ Crear **diagramas de entidad-relación** para modelar datos
- 🔄 Crear **diagramas de secuencia** para flujos de interacción
- 📐 Conocer los **símbolos UML** esenciales
- 📝 Escribir diagramas con **Mermaid** (sintaxis en texto)
- 🛠️ Usar **herramientas** como draw.io, Excalidraw y Lucidchart

Mapa de la unidad

Punto	Tema	Idea clave
01	¿Qué son los diagramas?	Comunicación visual del sistema
02	Modelo C4	4 niveles de arquitectura
03	Diagrama ER	Modelar base de datos
04	Diagrama de secuencia	Flujos de interacción
05	Símbolos UML	Referencia de notación
06	Mermaid	Diagramas en código de texto
07	Herramientas	draw.io, Excalidraw, Lucidchart
08	Template diagrams	Plantillas para tu proyecto
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 06 \(Mermaid\)](#) para empezar a crear diagramas en código
- **Si no tienes idea:** empieza por el [punto 01 \(¿Qué son?\)](#) para entender por qué importan

- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.7: IA como Copiloto

 *La IA no programa por ti, pero programa mejor contigo*

La inteligencia artificial ha llegado al desarrollo de software para quedarse. Pero no es un atajo mágico: es una herramienta que, usada bien, te hace más rápido y mejor. Usada mal, te genera código que no entiendes, bugs escondidos y problemas de seguridad. Esta unidad te enseña a usar la IA como un buen copiloto: que ayude, no que tome el volante.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué hace bien y qué hace mal la IA en desarrollo
-  Escribir prompts efectivos que obtienen código útil
-  Revisar y testear código generado por IA
-  Evitar problemas de seguridad al usar IA
-  Usar la IA de forma ética y responsable

Mapa de la unidad

Punto	Tema	Idea clave
01	Qué hace bien y mal la IA	Fortalezas y limitaciones reales
02	Prompts efectivos	Cómo escribir para obtener resultados
03	GitHub Copilot	Uso práctico del copiloto
04	Testing con IA	Cómo testear código generado
05	Seguridad con IA	Riesgos y cómo mitigarlos
06	Ética de la IA	Uso responsable
07	Errores comunes	Los fallos más frecuentes
08	Template de uso	Plantilla para documentar tu uso de IA
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 08 \(template\)](#) y documenta tu uso de IA
- **Si eres novato con IA:** empieza por el [punto 01 \(qué hace bien y mal\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.8: Despliegue

 *Tu proyecto no existe hasta que alguien puede usarlo en producción*

Un proyecto que solo funciona en localhost es un borrador. El despliegue es el paso que convierte tu código en un **producto real**: accesible desde cualquier navegador, con HTTPS, actualizaciones automáticas y sin downtime. Es lo que separa un proyecto de clase de algo profesional.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué es Docker y cómo empaquetar aplicaciones
-  Configurar pipelines CI/CD con GitHub Actions
-  Desplegar en plataformas reales (GitHub Pages, Netlify, Vercel, Railway)
-  Aplicar estrategias de zero-downtime
-  Seguir un checklist de despliegue completo

Mapa de la unidad

Punto	Tema	Idea clave
01	Qué es el despliegue	De localhost a producción
02	Docker	Empaquetar aplicaciones
03	CI/CD	Automatizar build y deploy
04	Plataformas	Dónde desplegar
05	Zero-downtime	Sin interrupciones
06	Checklist	Lista completa de pasos
07	Errores comunes	Qué falla y por qué
08	Template	Plantilla de despliegue
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 06 \(checklist\)](#) y sigue los pasos
- **Si no sabes qué es Docker:** empieza por el [punto 02 \(Docker\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.9: Monitorización

Desplegar tu app es solo el paso intermedio. Una vez en producción, necesitas **saber qué está pasando**: ¿cuántos usuarios tiene?, ¿hay errores?, ¿es rápida o lenta?, ¿se cae a veces?. La monitorización te da esas respuestas con datos, no con suposiciones.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Entender qué son los niveles de log y cómo usarlos
-  Definir las métricas clave de tu aplicación
-  Configurar Prometheus y Grafana como stack de monitorización
-  Crear alertas para detectar problemas antes de que los usuarios
-  Construir dashboards que muestren el estado de tu app

Mapa de la unidad

Punto	Tema	Idea clave
01	Qué son los logs	Niveles de log y estructura
02	Métricas clave	Latencia, errores, throughput
03	Prometheus + Grafana	El stack estándar
04	Alertas	Detectar problemas en tiempo real
05	Stack mínimo	Qué necesitas para empezar
06	Configuración	Winston, prom-client y más
07	Dashboard	Construir dashboards útiles
08	Template	Plantilla de configuración
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 05 \(stack mínimo\)](#) y configura lo básico

- **Si no sabes qué son los logs:** empieza por el [punto 01 \(logs\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

UD 2.10: Propuesta Final

 *Todo se une: el proyecto que demuestra lo que sabes hacer*

Esta es la unidad final. Aquí convergen todo lo aprendido: requisitos, SCRUM, Git, patrones, diagramas, IA, despliegue y monitorización. No es solo “hacer un proyecto”: es demostrar que puedes llevar una idea de la conceptualización a producción con metodología profesional.

Objetivos de la unidad

Al finalizar esta unidad serás capaz de:

-  Definir un proyecto Full-Stack completo con requisitos claros
-  Generar y seleccionar la mejor idea de proyecto
-  Elegir el stack tecnológico adecuado
-  Planificar el cronograma con hitos realistas
-  Preparar los entregables para la evaluación

Mapa de la unidad

Punto	Tema	Idea clave
01	Briefing final	Qué se espera del proyecto
02	Ideas finales	Generar opciones de proyecto
03	Selección	Elegir la mejor idea
04	Stack final	Elegir tecnologías
05	Features	Definir funcionalidades
06	Cronograma	Planificar el tiempo
07	Entregables	Qué entregar
08	Evaluación	Cómo se evalúa
09	Cierre	Mini-chequeo + resumen

¿Por dónde empiezo?

- **Si tienes prisa:** ve directamente al [punto 07 \(entregables\)](#) y prepara lo que hay que entregar
- **Si no tienes idea de proyecto:** empieza por el [punto 02 \(ideas\)](#)
- **Si quieres entender bien:** ve los puntos en orden del 01 al 09

Metodología

 *El día que entendieron que la metodología no era burocracia*

“¿Otra reunión?”, “¿Otra columna en el tablero?”, “¿Otra estimación?”. Al principio, todo lo que sonaba a “metodología” les parecía pérdida de tiempo. Querían programar, no hacer reuniones.

Hasta que llegó la semana antes de la entrega y descubrieron: - Una funcionalidad que daban por hecha no estaba empezada - Dos personas habían trabajado en lo mismo sin saberlo - Nadie sabía qué quedaba por hacer exactamente

La metodología no es burocracia. Es **no llegar en pánico al final**.

¿Qué metodología usar?

No hay una única respuesta. Depende de tu equipo y tu proyecto.

✅ Scrum (SCRUM Lite)

Sprints de 1-2 semanas. Reuniones cortas con propósito. Roles rotativos. Ideal para fechas de entrega fijas. Se trabaja a fondo en la UD 1.2 (SCRUM Lite) y la UD 2.2 (SCRUM Avanzado).

✅ Kanban

Flujo continuo sin sprints. Límites de trabajo en curso (WIP). Menos reuniones. Ideal para mantenimiento y mejoras. También se cubre en la UD 1.2.

SCRUM (adaptado)

Para equipos de 2-4 personas, con sprints de 1-2 semanas.

 Esta sección resume lo esencial. En profundidad, verás **SCRUM Lite** en la [UD 1.2](#) y **SCRUM Avanzado** (métricas, burndown, retrospectivas) en la [UD 2.2](#).

Qué hacer: - Sprint planning al empezar cada sprint (qué haremos, quién lo hará) - Daily (15 min, de pie o sentados, pero rápidos) - Sprint review al terminar (qué se ha conseguido) - Retrospectiva (qué mejorar para el próximo sprint)

Qué NO hacer: - Scrum Master oficial (rotad el rol) - Ceremonias de 2 horas - Estimar en puntos de historia (usad horas o días) - Sprint backlog inmutable (se puede renegociar)

Kanban

Para equipos que prefieren fluidez continua sin sprints fijos.

Qué hacer: - Tablero con columnas: Pendiente → En curso → Revisión → Hecho - Límite de tareas en “En curso” (máximo 2 por persona) - Flujo continuo sin fechas fijas

Ventaja: más flexible, menos reuniones. **Desventaja:** menos estructura, fácil procrastinar.

Git Flow

No es solo para el código. También organiza el trabajo:

Rama	Propósito
main	Código estable y desplegado
develop	Integración de funcionalidades
feature/*	Una funcionalidad nueva (una por persona)
hotfix/*	Corrección urgente

 En profundidad: [UD 2.3 — Git Avanzado](#).

El ciclo del proyecto

Sea cual sea la metodología, el proyecto sigue este ciclo:

**  Planificar →

→ → **  Ejecutar →

→ → **  Revisar →

→ → **  Aprender → →

Cada iteración (semana, sprint) deberías: 1. Elegir qué hacer 2. Hacerlo 3. Comprobar que funciona 4. Reflexionar sobre lo aprendido

Sin el paso 4, repites los mismos errores. Sin el paso 1, trabajas sin dirección.

Caso 1: El equipo que no planificaba

Un equipo de 3 personas decidió que “la metodología era pérdida de tiempo”. Se pusieron a programar directamente desde el día 1.

Resultado: - A 3 días de la entrega, descubrieron que dos personas habían implementado la misma funcionalidad - Una feature crítica no se había empezado - Tuvieron que dormir 4 horas al día para llegar

Lección: 30 minutos de planificación al inicio de la semana habrían ahorrado 3 días de caos.

Ejemplo de sprint de 2 semanas

Semana 1

Día	Mañana	Tarde
Lunes	Sprint planning (30 min)	Desarrollo
Martes	Daily (15 min)	Desarrollo
Miércoles	Daily (15 min)	Desarrollo
Jueves	Daily (15 min)	Desarrollo
Viernes	Daily (15 min)	Revisión parcial

Semana 2

Día	Mañana	Tarde
Lunes	Daily (15 min)	Desarrollo
Martes	Daily (15 min)	Desarrollo
Miércoles	Daily (15 min)	Desarrollo
Jueves	Daily (15 min)	Últimas pruebas
Viernes	Sprint review + Retro (45 min)	 Descanso

Total de reuniones: ~**3 horas en 2 semanas** (menos del 5% del tiempo).

Errores comunes

- **Sobreplanificar:** perder 2 días planificando un sprint de 1 semana
- **Infraplanificar:** empezar a programar sin saber qué hace cada uno
- **No hacer retrospectiva:** repetir los mismos errores sprint tras sprint
- **Daily que se alarga:** las daily son de 15 min, no de 45

Herramientas recomendadas

Herramienta	Para qué
Trello / GitHub Projects	Tablero Kanban
Notion / Confluence	Documentación compartida
Discord / Slack	Comunicación diaria
Google Calendar	Reuniones y fechas límite
Git + GitHub / GitLab	Control de versiones

Herramientas

 *El día que se dieron cuenta de que no necesitaban 20 herramientas*

El equipo empezó con entusiasmo: un chat para hablar, otro para compartir archivos, un tablero para tareas, un documento para ideas, otro documento para la memoria, un repositorio, un gestor de contraseñas...

A las dos semanas, la información estaba dispersa en 8 sitios distintos. Nadie recordaba dónde estaba cada cosa. Perdían más tiempo buscando información que creándola.

Menos herramientas. Mejor organizadas.

Herramientas esenciales

No necesitas más de 4-5 herramientas bien usadas.

1. Repositorio de código (Git + GitHub/GitLab)

Tu repositorio no es solo para el código. También es para: - Documentación técnica (README, Wiki) - Issues (tareas, bugs, ideas) - Pull requests (revisión de código) - Projects (tablero Kanban integrado)

Tip: usa GitHub Projects como tablero. Así tienes código y tareas en el mismo sitio. Una herramienta menos que gestionar.

2. Comunicación (Discord, WhatsApp, Slack)

Elegid una. Solo una. No combinéis WhatsApp para “urgente” y Discord para “lo demás”. Todo en el mismo sitio.

Consejo: cread canales o grupos separados para: - General (anuncios, dudas generales) - Técnico (problemas de código) - Off-topic (memes, no interrumpir a los demás)

3. Documentación (Markdown en el repo)

La memoria del proyecto, las decisiones técnicas, los diagramas. Todo en un sitio accesible para todos.

Recomendación: usad Markdown en el propio repositorio. Así la documentación viaja con el código y tenéis control de versiones.

4. Gestión de tareas (GitHub Projects, Trello, Notion)

Un tablero con tres columnas basta:

** 📄 Por hacer →

→ → ** ⚙ En proceso →

→ → ** ✅ Hecho → →

Si necesitas más, añade “Revisión” o “Bloqueado”. Pero no te pases. Cuantas más columnas, más tiempo pierdes moviendo tarjetas.

📄 Caso 2: El día que consolidaron sus herramientas

Un equipo usaba: - WhatsApp para hablar - Trello para tareas - Google Docs para documentación - GitHub para código - Figma para diseños - Drive para assets

El problema: cada vez que necesitaban algo, no sabían dónde buscar. La información estaba fragmentada.

La solución: - Movieron todo a GitHub (Issues + Projects + Wiki) - Markdown para documentación (dentro del repo) - Discord para comunicación (integrado con GitHub) - Figma solo para prototipos iniciales

Resultado: 4 herramientas → 3. La información, centralizada.

5. IDE (VS Code recomendado)

VS Code es el estándar por su ecosistema de extensiones:

Extensión	Para qué
ESLint / Prettier	Código limpio y formateado
GitLens	Historial y blame en línea
GitHub Pull Requests	Revisar PRs sin salir del IDE
Live Share	Programar en pareja en remoto
Docker	Gestionar contenedores
Thunder Client	Probar APIs (alternativa a Postman)

Herramientas complementarias

Herramienta	Para qué	Alternativas
Postman / Insomnia	Probar APIs	Hoppscotch
Draw.io / Figma	Diagramas y prototipos	Excalidraw, Mermaid
Docker	Contenedores	Podman
Vercel / Netlify	Despliegue frontend	Render, GitHub Pages
UptimeRobot	Monitorizar que la app responde	—

 Estas herramientas se trabajan en las UD's: - [UD 1.4 — Elección de Tecnologías](#) - [UD 2.6 — Diagramas](#) - [UD 2.8 — Despliegue](#) - [UD 2.9 — Monitorización](#)

Flujo de trabajo típico

1. Abres una Issue en GitHub → tarea pendiente
2. Mueves la Issue a "En proceso" → estás trabajando
3. Creas una rama feature/ desde develop → código
4. Abres un Pull Request → revisión del compañero
5. Mergeas a develop → integración
6. Cierras la Issue → 

Lo que NO necesitas

- Un IDE distinto cada uno (cada uno que use el que quiera)
- Una herramienta de gestión de proyectos solo porque está de moda
- Un servicio cloud que nadie sabe configurar
- Una base de datos externa cuando SQLite local es suficiente
- Un canal de comunicación distinto para cada tema

Recursos

 *El día que Stack Overflow era su mejor amigo sin Internet*

“No me funciona este código”, dijo una voz en el grupo. “Búscalo en Google”, respondió otra.

Buscaron. Encontraron una respuesta en Stack Overflow de 2015. La probaron. No funcionaba. Buscaron otra. Esa sí.

Internet es tu mejor recurso. Pero hay que saber buscar.

Cómo buscar ayuda técnica

Busca en el orden correcto

1. **Documentación oficial** (la fuente más fiable)
2. **Stack Overflow** (problemas comunes ya resueltos)
3. **GitHub Issues** (bugs conocidos, soluciones de la comunidad)
4. **Tutoriales / blogs** (guías paso a paso)
5. **Pregunta a un compañero o al profesor** (cuando lleves 30 min atascado)

Cómo preguntar bien

Cuando preguntes (en Stack Overflow, al profesor, a un compañero), sé específico:

✗ “No me funciona el login” ✓ “Al enviar el formulario de login con email y contraseña válidos, recibo un error 500 en la consola del servidor. El código relevante es...”

Incluye siempre: - Qué esperabas que pasara - Qué pasa realmente - El código relevante (no todo el archivo) - Los mensajes de error completos

Caso 3: El día que aprendieron a buscar

Un alumno llevaba 3 horas atascado con un error de conexión a base de datos. Había probado “cosas” sin saber bien qué hacía.

Su proceso: - Pegó el error en Google - Probó la primera respuesta (no funcionó) - Probó la segunda (tampoco) - Se rindió y preguntó al profesor

El profesor hizo: - Copió el error exacto en Google - Encontró un GitHub Issue con el mismo error - La solución era cambiar una línea de configuración

Lección: el error estaba documentado en GitHub Issues, pero él no sabía buscar allí. Aprender a buscar es tan importante como aprender a programar.

Referencias por tema

Frontend

- [MDN Web Docs](#) — la biblia del desarrollo web
- [CSS-Tricks](#) — guías visuales de CSS
- Documentación oficial de React / Vue / Svelte

Backend

- [Express.js](#) guía oficial
- [Node.js](#) documentación de la API
- [Spring Boot](#) guía oficial

Bases de datos

- Documentación oficial de PostgreSQL / MySQL
- [SQL Tutorial](#) (w3schools, para empezar)

- [Pro Git](#) — libro gratuito y completo
- [Oh Shit, Git!?!](#) — cuando algo sale mal

Docker

- [Docker Curriculum](#) — tutorial interactivo
- Documentación oficial de Docker

IA como recurso de aprendizaje

►  ¿Puedo usar ChatGPT/Claude para el proyecto?

El recurso más infravalorado: preguntar

No estás solo en el proyecto. Tu profesor, tus compañeros y la comunidad son recursos que muchos no usan por miedo o pereza. Pregunta:

- ¿Voy por buen camino?
- ¿Esta decisión técnica tiene sentido?
- ¿Cómo harías tú esto?
- ¿Qué priorizarías ahora?

Una consulta de 5 minutos a una persona con experiencia puede ahorrarte 2 días de trabajo. Preguntar bien (con el error concreto y el contexto) es una habilidad profesional en sí misma.

 Sobre cómo usar la IA como recurso de aprendizaje, verás más en la [UD 2.7 — IA como Copiloto](#).

Licencia

 *Compartir es aprender*

Este material didáctico y todos los materiales asociados (apuntes, plantillas, ejemplos) se publican bajo la licencia **Creative Commons Atribución-CompartirIgual 4.0 Internacional (CC BY-SA 4.0)**.

Eres libre de

- **Compartir** — copiar y redistribuir el material en cualquier medio o formato
- **Adaptar** — remezclar, transformar y crear a partir del material

Bajo las siguientes condiciones

- **Atribución** — debes reconocer la autoría de forma adecuada, proporcionar un enlace a la licencia e indicar si se han realizado cambios
- **CompartirIgual** — si remezclas, transformas o creas a partir del material, deberás difundir tus contribuciones bajo la **misma licencia** que el original

Autoría

Sergi Garcia Barea — CC BY-SA 4.0



CC BY-SA 4.0

Más información en: <https://creativecommons.org/licenses/by-sa/4.0/>

? Preguntas Frecuentes

 *Las dudas que todo el mundo tiene*

Da igual el curso: siempre surgen las mismas preguntas. Aquí tienes las respuestas para que no pierdas tiempo ni te lleves sorpresas.

Sobre el proyecto

► ? ¿Puedo cambiar de idea a mitad de proyecto?

► ? ¿Qué pasa si mi equipo se rompe?

► ? ¿Puedo hacer el proyecto individual?

► ? ¿Qué pasa si el profesor rechaza mi idea?

► ? ¿Puedo usar inteligencia artificial?

Sobre la evaluación

► ? ¿Qué peso tiene cada unidad didáctica?

► ? ¿Qué pasa si no entrego una UD a tiempo?

► ? ¿La presentación oral cuenta?

► ? ¿Se evalúa el trabajo en equipo?

Sobre el código

► ? ¿Puedo usar código de internet?

► ? ¿Cuánto código tengo que escribir?

► ? ¿Tengo que hacer tests?

► ? ¿Qué pasa si mi código no compila?

Sobre la documentación

► ? ¿La memoria se entrega al final?

► ? ¿Qué extensión debe tener?

► ? ¿Tengo que hacer una memoria en PDF?

► ? ¿Tienes otra duda?



Ideas para Proyectos

 *El día que no se les ocurría nada*

“Vale, tenéis que hacer un proyecto. Ideas, ideas...” Silencio en la clase. Nadie sabía qué hacer.

No hace falta una idea revolucionaria. Hace falta una idea que **resuelva un problema real**, aunque sea pequeño.

Cómo elegir una buena idea

	Buena idea	Mala idea
Alcance	Resuelve un problema concreto	“Una red social para todo”
Usuario	Tiene un usuario definido	“Es para todo el mundo”
Tecnología	Usa lo que sabes	“Necesito aprender IA en una semana”
Motivación	Te interesa el tema	“Es lo único que se me ocurre”

Ideas para PI1

Gestión y organización

- **Gestor de tareas** para un equipo pequeño
- **Control de inventario** para un pequeño negocio
- **Sistema de reservas** para una peluquería, taller o pista deportiva
- **Gestor de gastos** personales o de un viaje
- **Organizador de eventos** (bodas, cumpleaños, conferencias)

Educación

- **Plataforma de apuntes** compartidos entre estudiantes
- **Gestor de trabajos** para un instituto (entrega, corrección, notas)
- **Sistema de tutorías** (reservar y gestionar sesiones)
- **Generador de horarios** de clase
- **App de Flashcards** para estudiar

Comunidad

- **Directorio de recursos** de un barrio (talleres, asociaciones)
- **Red de trueque** de objetos entre vecinos
- **Gestor de incidencias** para una comunidad de vecinos
- **App para compartir coche** entre compañeros de clase

Tiempo real / Datos

- **Tablero Kanban** para equipos pequeños
- **Gestor de hábitos** y seguimiento personal
- **Sistema de encuestas** con resultados en tiempo real

- **Gestor de recetas** con filtros por ingredientes

Cómo validar tu idea

Antes de empezar a programar, valida que tu idea tiene sentido:

1. **Háblala con 3 personas:** ¿les parece útil? ¿la usarían?
2. **Búscala en Google:** ¿ya existe algo así? ¿qué harías diferente?
3. **Pregunta a tu profesor o a la comunidad:** ¿es viable para el tiempo que tenemos?
4. **Recorta alcance:** ¿cuál es la versión más pequeña que sigue siendo útil?

 En la [UD 1.7 — Propuesta de Proyecto](#) aprenderás a transformar estas ideas en una propuesta formal (briefing, alcance y cronograma).

Ideas para PI2

En PI2 partís del proyecto de PI1. Las ideas se centran en **evolucionar**:

- Añadir autenticación con Google/GitHub
- Integrar pagos simulados (Stripe test)
- Añadir mapas (tiendas cercanas, rutas)
- Implementar chat en tiempo real (WebSockets)
- Dashboard con gráficos y estadísticas
- Exportar datos a PDF/CSV
- Notificaciones por email
- Versión móvil responsive avanzada
- Panel de administración
- Tests automatizados y CI/CD

Y si no tienes ni idea

Empieza por un problema que **tú mismo tengas**:

- “Necesito organizar mis apuntes”
- “Mi madre necesita gestionar las citas de su negocio”
- “En mi club deportivo no tienen web”

- “Odio hacer la lista de la compra y siempre se me olvida algo”

El mejor proyecto es el que te sirve a ti. Porque si a ti te sirve, lo harás con ganas. Y si lo haces con ganas, sale mejor.